Appendix C - Random Database Connection Closed Exceptions

These can occur when one request gets a db connection from the connection pool and closes it twice. When using a connection pool, closing the connection just returns it to the pool for reuse by another request, it doesn't close the connection. And Tomcat uses multiple threads to handle concurrent requests. Here is an example of the sequence of events which could cause this error in Tomcat:

1. Request 1 running in Thread 1 gets a db connection.
2. Request 1 closes the db connection.
3. The JVM switches the running thread to Thread 2
4. Request 2 running in Thread 2 gets a db connection(the same db connection just closed by Request 1).
5. The JVM switches the running thread back to Thread 1
6. Request 1 closes the db connection a second time in a finally block.
7. The JVM switches the running thread back to Thread 2
8. Request 2 Thread 2 tries to use the db connection but fails because Request 1 closed it.

Here is an example of properly written code to use a db connection obtained from a connection pool:

```
Connection conn = null;
Statement stmt = null;  // Or PreparedStatement if needed
ResultSet rs = null;
try {
  conn = ... get connection from connection pool ...
  stmt = conn.createStatement("select ...");
  rs = stmt.executeQuery();
  ... iterate through the result set ...
  rs.close();
  rs = null;
  stmt.close();
  stmt = null;
  conn.close(); // Return to connection pool
  conn = null;  // Make sure we don't close it twice
} catch (SQLException e) {
  ... deal with errors ...
} finally {
  // Always make sure result sets and statements are closed,
  // and the connection is returned to the pool
  if (rs != null) {
    try { rs.close(); } catch (SQLException e) { ; }
    rs = null;
  }
  if (stmt != null) {
    try { stmt.close(); } catch (SQLException e) { ; }
    stmt = null;
  }
  if (conn != null) {
    try { conn.close(); } catch (SQLException e) { ; }
    conn = null;
  }
}
```